

Appendix

Interaction Classifier Architecture

The following Appendix provides a detailed description of the interaction classifier architecture. Figures 1 and 2 show the architectures for force and point cloud encoders, respectively. Further, Figure 3 shows the multimodal transformer architecture for fusing both modality representations, followed by the model classification head shown in Figure 4.

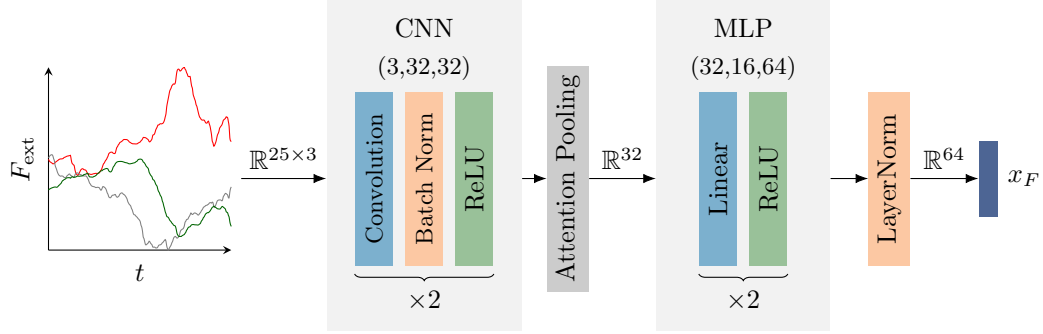


Figure 1: Force history encoder.

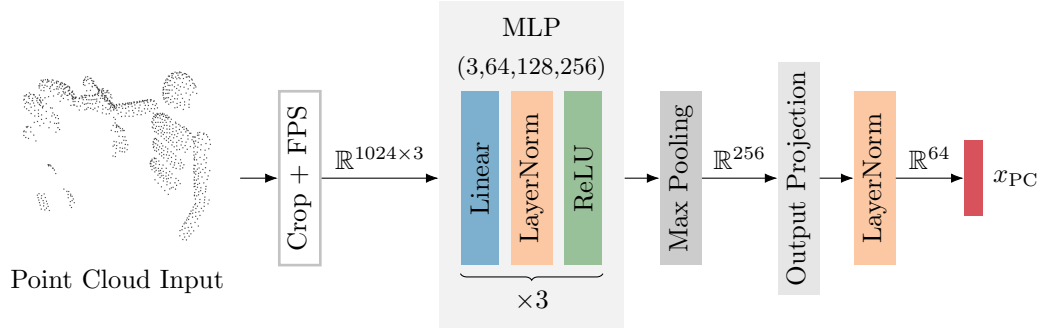


Figure 2: 3D Point Cloud Encoder [4].

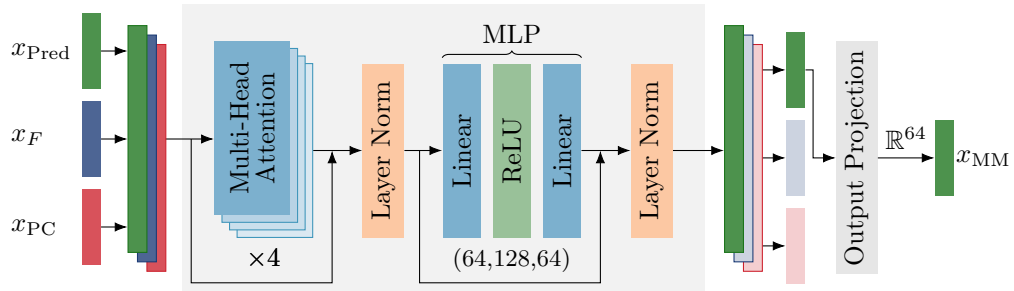


Figure 3: Multimodal Transformer.

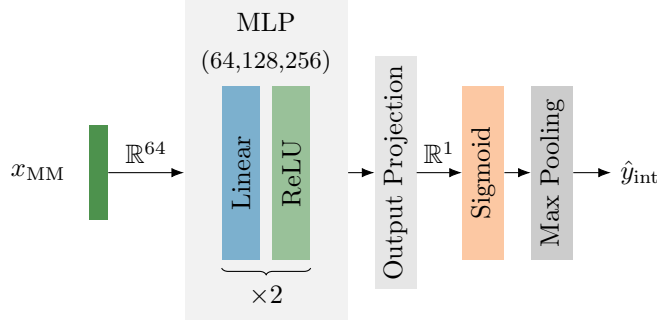


Figure 4: Classification Head.

Training Information

Tables 1 and 2 show training parameters for encoders and full model pretraining, respectively. Table 3 shows model fine-tuning training parameters. For all training stages, we utilized PyTorch Library [3] and workstation with an Intel[®] Core[™] Ultra 9 285K CPU and NVIDIA[®] GeForce RTX[™] 5090 GPU.

	Learning Rate (η)	Epochs	$\mathcal{L}_{\text{train}}$	$\mathcal{L}_{\text{val}}$	Time (min)
Force Encoder	0.0001	5000	0.162	0.201	1:53
Point Cloud Encoder	0.0001	1000	0.0455	0.0951	4:08

Table 1: Training parameters for encoder pretraining.

Encoders Learning Rate (η)	Transformer & Head Learning Rate (η)	Epochs	$\mathcal{L}_{\text{train}}$	$\mathcal{L}_{\text{val}}$	Time (min)
0.00001	0.0001	1000	0.00737	0.0839	4:25

Table 2: Training parameters for model pretraining.

Dataset Composition (sim/real)	Learning Rate (η)	Epochs	$\mathcal{L}_{\text{train}}$	$\mathcal{L}_{\text{val}}$	Time (min)
10,000/25,000	0.00005	1000	0.0185	0.0479	11:57

Table 3: Training parameters for model fine-tuning.

Finally, we fine-tune the classifier with a mix of simulation-based and real data to facilitate sim-to-real transfer, while maintaining the generalization properties of simulation-based pretraining. However, since the bimanual setup is not within the training data, we decided to fine-tune the network for this experiment only on the real-robot dataset to better transfer to unseen morphologies. The classifier used in the bimanual experiment thus has the same architecture and pretraining, but instead of using a mix of simulated and real data for fine-tuning, it uses only the real data subset. We plan to create specialized model checkpoints tailored to specific morphologies in our future work, enabling more robust framework transfer.

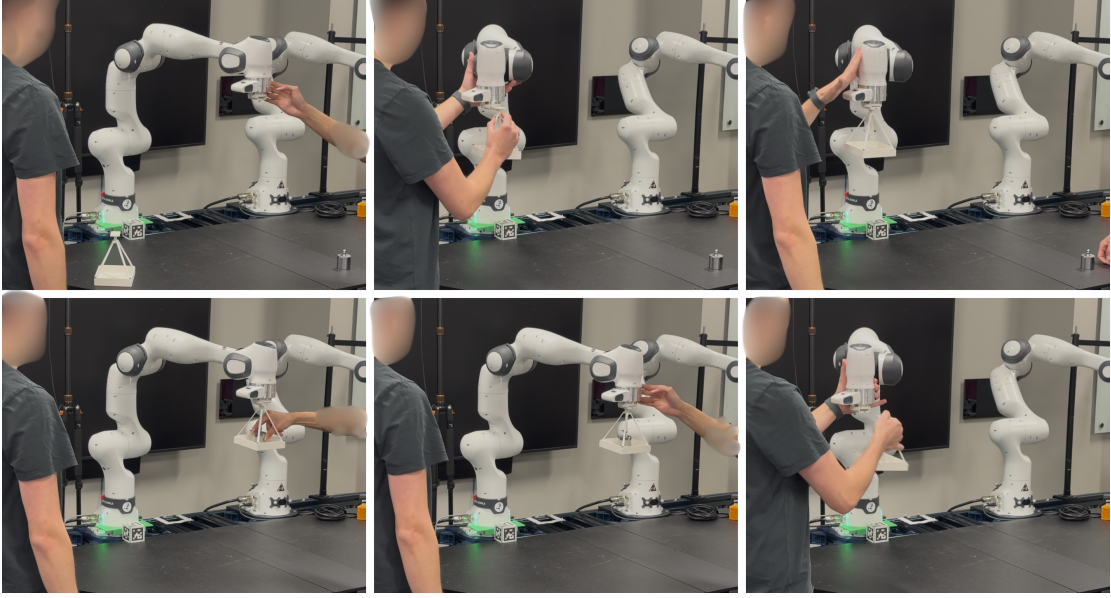


Figure 5: Intent estimation using interaction force estimation. As shown, the framework provides informative estimates of interactions, enabling natural target switching upon detecting human intent.

Hidden Markov Model-based Target Belief filter

To examine solver’s accuracy in estimating interaction forces, we conducted an intent estimation experiment, where the robot utilized $\hat{F}_{\text{int}}$ as the input to a Hidden Markov Model-inspired target belief filter, which estimated to which of the predefined targets the robot should move next by evaluating likelihood function conditioned on the alignment of the interaction force with the distance vector from the robot’s end-effector to the given target. To achieve that, we define n targets for the robot, a belief vector $b_k \in \mathbb{R}^n$, and the transition matrix $M \in \mathbb{R}^{n \times n}$. The state transition is defined as

$$b_{k|k-1} = M^\top b_{k-1} \quad (1)$$

where each entry of M is defined as $M_{ii} = 1 - \alpha$ (stay at target) and $M_{ij} = \frac{\alpha}{n-1}$ (change target), with transition tuning parameter $\alpha = 0.0001$. Then, we define the observation model as the cosine alignment s_k between the interaction force vector F_{int} and the unit vector from the end-effector to a given target u_k as

$$s_k = \frac{F_{\text{int}} \cdot u_k}{\|F_{\text{int}}\| + \epsilon} \quad (2)$$

where $\epsilon = 10^{-6}$. Further, we formulate the likelihood model as

$$L_k = \exp(\beta s_k \|F_{\text{int}}\|) (b_{k-1} + \epsilon)^{\lambda_k} \quad \lambda_k = \exp\left(-\frac{\|F_{\text{int}}\|}{F_0 + \epsilon}\right) \quad (3)$$

with $\beta = 0.001$, where the first term of the likelihood reinforces the target with the strongest force alignment, and the second term reinforces the current target prediction when no interaction forces are present, or they are smaller than $F_0 = 0.3N$. The target belief is then updated as

$$b_k = L_k b_{k|k-1} \quad (4)$$

and the robot proceeds to the target based on its firmest current belief. This formulation allows us to verify the quality of the estimated interaction force. During the evaluation, the framework provided informative estimates of interaction forces, enabling a collaborative human-robot-human transportation task, as shown in Figure 5.

Passivity Properties of the Variable Impedance Controller

To verify the passivity of the proposed adaptive controller, we examine the Variable Impedance Controller's passivity properties, based on the passivity proof proposed in [1]. Let us restate our task-space dynamics

$$M_x(q)\ddot{x} + C_x(q, \dot{q})\dot{x} + G_x(q) = W_c + F_{\text{ext}} \quad (5)$$

where M_x , C_x , and G_x are task-space mass, Coriolis and gravity matrices, respectively, and W_c is the passive impedance controller

$$W_c(x) = G_x(q) - \gamma_d(\cdot)D(x)(\dot{x} - f(x)), \quad \tau_c = J(q)^\top W_c \quad (6)$$

Inserting the controller into the system, we obtain the following closed-loop dynamics

$$M_x(q)\ddot{x} + C_x(q, \dot{q})\dot{x} = -\gamma_d(\cdot)D(x)(\dot{x} - f(x)) + F_{\text{ext}} \quad (7)$$

We can design the damping function $D(x)$ to dissipate energy in orthogonal directions to $f(x)$ formulated as $D(x) = V(x)\Lambda(x)V(x)^\top$:

$$V(x) = [e_1(x) \quad e_2(x)], \quad e_1(x) = \frac{f(x)}{\|f(x)\|}, \quad e_1(x)^\top e_2(x) = 0, \quad \Lambda(x) = \begin{bmatrix} \lambda_1(x) & 0 \\ 0 & \lambda_2(x) \end{bmatrix} \quad (8)$$

as presented in[2], where $\lambda_i(x) \geq 0$ sets the impedance in orthogonal directions of $Q(x)$ basis. This simplifies the controller to:

$$M_x(q)\ddot{x} + (C_x(q, \dot{q}) + \gamma_d(\cdot)D(x))\dot{x} - \gamma_d(\cdot)\lambda_1 f(x) = F_{\text{ext}} \quad (9)$$

The stability and passivity of the robot's closed-loop dynamics can be proven with the following energy storage function:

$$S(x, \dot{x}) = \frac{1}{2}\dot{x}^\top M_x \dot{x} + \gamma_d(\cdot)\lambda_1 \mathcal{V}(x) \quad (10)$$

which includes the robot's kinetic energy and the potential energy injected by the controller term, and accounts for the variable impedance parameter, which also scales the dynamical system in the controller formulation. The potential energy depends solely on the eigenvalue λ_1 and the Lyapunov function $\mathcal{V}(x)$ used to ensure asymptotic stability of $f(x)$, which, for this analysis, we assume to be conservative $f(x) = -\nabla \mathcal{V}(x)$. The time derivative of the energy storage function is,

$$\begin{aligned} \dot{S}(x, \dot{x}) &= \dot{x}^\top M_x \ddot{x} + \frac{1}{2}\dot{x}^\top \dot{M}_x \dot{x} + \gamma_d(\cdot)\lambda_1 \nabla \mathcal{V}^\top \dot{x} + \dot{\gamma}_d(\cdot)\lambda_1 \mathcal{V} \\ &= \dot{x}^\top (F_{\text{ext}} - (C_x + \gamma_d(\cdot)D(x))\dot{x} + \gamma_d(\cdot)\lambda_1 f(x)) + \frac{1}{2}\dot{x}^\top \dot{M}_x \dot{x} + \gamma_d(\cdot)\lambda_1 \nabla \mathcal{V}(x) + \dot{\gamma}_d(\cdot)\lambda_1 \mathcal{V} \\ &= \dot{x}^\top F_{\text{ext}} - \dot{x}^\top \gamma_d(\cdot)D(x)\dot{x} + \frac{1}{2}\dot{x}^\top \underbrace{(\dot{M}_x - 2C_x)}_{=0} \dot{x} + \gamma_d(\cdot)\lambda_1 (f(x) + \nabla \mathcal{V}(x)) + \dot{\gamma}_d(\cdot)\lambda_1 \mathcal{V} \\ &= \dot{x}^\top F_{\text{ext}} - \dot{x}^\top \gamma_d(\cdot)D(x)\dot{x} + \dot{\gamma}_d(\cdot)\lambda_1 \mathcal{V} \end{aligned} \quad (11)$$

which implies that the system will be passive w.r.t the port $(F_{\text{ext}}, \dot{x})$ if

$$\dot{x}^\top F_{\text{ext}} - \dot{x}^\top \gamma_d(\cdot)D(x)\dot{x} + \dot{\gamma}_d(\cdot)\lambda_1 \mathcal{V}(x) \leq \dot{x}^\top F_{\text{ext}} \quad (12)$$

which simplifies to

$$\dot{\gamma}_d(\cdot)\lambda_1 \mathcal{V}(x) \leq \dot{x}^\top \gamma_d(\cdot)D(x)\dot{x}. \quad (13)$$

Since $\lambda_1 \mathcal{V}(x) \geq 0$, this condition can be equivalently written as

$$\dot{\gamma}_d(\cdot) \leq \frac{\dot{x}^\top \gamma_d(\cdot)D(x)\dot{x}}{\lambda_1 \mathcal{V}(x) + \epsilon} \quad (14)$$

where ϵ is a small constant guaranteeing numerical stability. Since we update γ_d recursively, we can approximate its derivative as

$$\dot{\gamma}(\cdot) = \frac{\gamma_{d,k} - \gamma_{d,k-1}}{\Delta t} \quad (15)$$

where Δt is the time difference between parameter computation. Substituting this in the equation results in the following expression

$$\gamma_{d,k} \leq \gamma_{d,k-1} + \Delta t \frac{\dot{x}^\top \gamma_{d,k-1} D(x_{k-1}) \dot{x}}{\lambda_1 \mathcal{V}(x_{k-1}) + \epsilon} \quad (16)$$

which bounds the maximum increase in γ_d over the recursive update. By enforcing this condition on the update, we can guarantee that the system will be passive w.r.t the port $(F_{\text{ext}}, \dot{x})$ when $F_{\text{ext}} \neq 0$, and stable when $F_{\text{ext}} = 0$. Thus, we notice that passivity conditions reduce to

$$\begin{cases} \dot{\gamma}_d \leq 0 & \rightarrow (\text{passive}) \\ \dot{\gamma}_d > 0 \quad \wedge \quad \dot{\gamma}_d(\cdot) \leq \frac{\dot{x}^\top \gamma_d(\cdot) D(x) \dot{x}}{\lambda_1 \mathcal{V}(x) + \epsilon} & \rightarrow (\text{passive}) \\ \dot{\gamma}_d > 0 \quad \wedge \quad \dot{\gamma}_d(\cdot) > \frac{\dot{x}^\top \gamma_d(\cdot) D(x) \dot{x}}{\lambda_1 \mathcal{V}(x) + \epsilon} & \rightarrow (\text{not passive}) \end{cases} \quad (17)$$

meaning that the system will always remain passive when reducing the variable impedance parameter γ_d . This is crucial because the case occurs when human contact is detected, which is when we do not want to break the system's passivity.

While we are always able to enforce the passivity on the system, we noticed that due to the small gradient values, the time that it takes to increase the impedance gains to reach a reactive state after the contact is no longer present increases from about 0.1 seconds to about 76 seconds, which is too slow for most of the systems to maintain high tracking accuracy. Therefore, we utilize the filtered contact estimation, which applies a safety buffer, delaying the increase in impedance gain after the human contact is no longer present. While this solution results in the injection of energy into the system, effectively breaking passivity, it does not happen when a human is in contact with the robot, having no negative impact on the interaction.

We can further examine passivity properties by analyzing the time-derivative of the energy storage function, $\dot{S}(x, \dot{x})$, given as

$$\dot{S}(x, \dot{x}) = \dot{x}^\top F_{\text{ext}} - \dot{x}^\top \gamma_d(\cdot) D(x) \dot{x} + \dot{\gamma}_d(\cdot) \lambda_1 \mathcal{V}. \quad (18)$$

Knowing that passivity is maintained when $\dot{S} \leq \dot{x}^\top F_{\text{ext}}$, we define power balance term of the controller

$$P_{\text{VIC}} = \dot{\gamma}_d(\cdot) \lambda_1 \mathcal{V} - \dot{x}^\top \gamma_d(\cdot) D(x) \dot{x} \quad (19)$$

which implies that the system preserves passivity if $P_{\text{VIC}} \leq 0$. As shown in Figure 6, P_{VIC} exceeds 0 only at the short instant when we increase the variable impedance parameter. However, since this power injection is instantaneous and absent during human-robot interaction, it does not negatively affect the interaction.

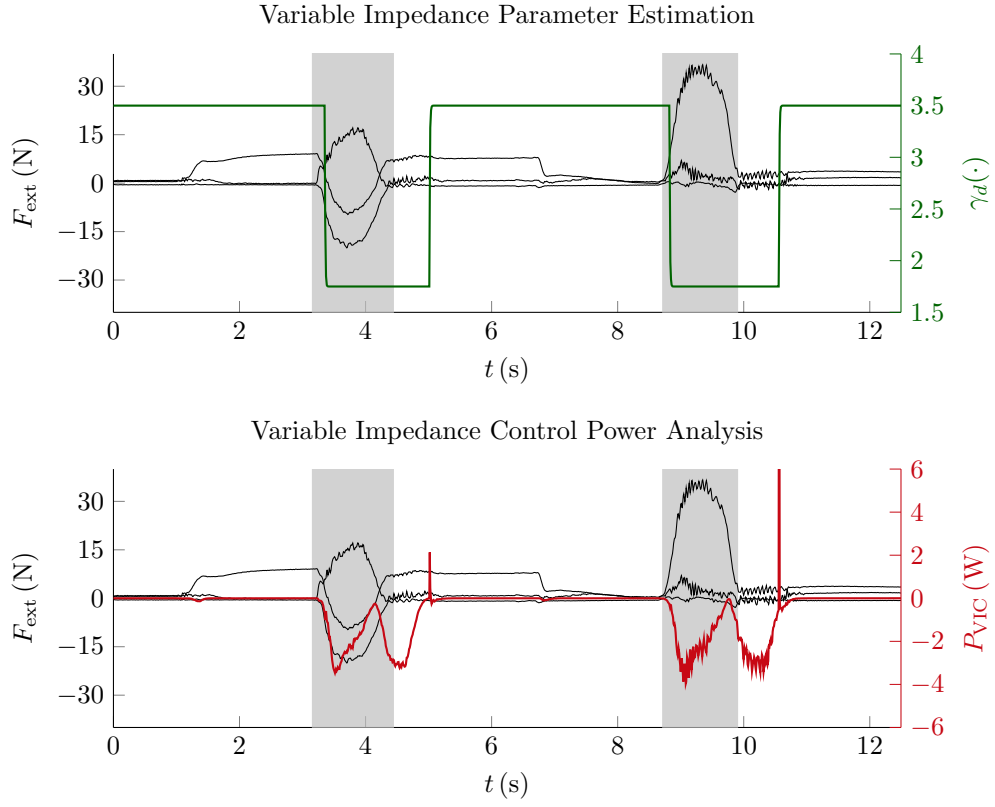


Figure 6: Time history of the variable impedance parameter and P_{VIC} when the adaptive controller detects both loads and perturbations. Gray shaded regions denote human perturbations. The maximum recorded P_{VIC} was observed to be ~ 25 W at the second spike shown in the plot.

References

- [1] Mateusz Jaszczuk and Nadia Figueroa. Rapid mismatch estimation via neural network informed variational inference. In *9th Annual Conference on Robot Learning*, 2025.
- [2] Klas Kronander and Aude Billard. Passive interaction control with dynamical systems. *IEEE Robotics and Automation Letters*, 1(1):106–113, 2015.
- [3] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019.
- [4] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. In *Proceedings of Robotics: Science and Systems (RSS)*, 2024.